

PROGRAMAÇÃO EM HASKELL



Capítulo 3 - Tipos e Classes

O que é um tipo?

É um nome para uma coleção de valores relacionados

Por exemplo, em Haskell, o tipo básico

`Bool`

contém os dois valores lógicos

`False`

`True`

Erros de tipo

A aplicação de uma função a um ou mais argumentos do tipo errado é chamada de **erro de tipo**

```
> 1 + False  
error ...
```

1 é um número e **False** é um valor lógico, mas **+** requer dois números

Tipos em Haskell

- Se avaliar uma expressão **e** produz um valor do tipo **t**, então **e** tem o tipo **t**, escrito

e :: t

- Toda **expressão bem formada** tem um tipo, que pode ser calculado automaticamente em tempo de compilação, usando um processo chamado **inferência de tipo**

- Todos os **erros de tipo** são detectados em **tempo de compilação**, tornando os programas mais seguros e rápidos, evitando a necessidade de verificações de tipo em tempo de execução
- No GHCi, o comando **:type** calcula o tipo de uma expressão, sem avaliá-la

```
> not False  
True
```

```
> :type not False  
not False :: Bool
```

Tipos básicos

Haskell tem uma série de tipos básicos, incluindo

`Bool`

- valores lógicos

`Char`

- caracteres únicos

`String`

- cadeias de caracteres

`Int`

- números inteiros

`Float`

- números de ponto flutuante

Tipos lista

Uma **lista** é uma sequência de elementos de um **mesmo tipo**

```
[False,True,False] :: [Bool]
```

```
['a','b','c','d'] :: [Char]
```

Em geral

[t] é o tipo de listas com elementos do tipo **t**

Nota

- O tipo de lista não diz nada sobre seu tamanho

```
[False,True] :: [Bool]
```

```
[False,True,False] :: [Bool]
```

- O tipo dos elementos é irrestrito, por exemplo, pode-se ter listas de listas

```
[['a'],['b','c']] :: [[Char]]
```


Tipos tupla

Uma **tupla** é uma sequência finita de componentes de tipos possivelmente diferentes

```
(False, True) :: (Bool, Bool)
```

```
(False, 'a', True) :: (Bool, Char, Bool)
```

Em geral

$(t_1, t_2, \dots, t_n)$ é o tipo de n -tupla, com aridade n , cujos i -ésimos componentes têm tipo t_i para qualquer i em $1..n$

Note

- O tipo de uma tupla codifica sua aridade

```
(False, True) :: (Bool, Bool)
```

```
(False, True, False) :: (Bool, Bool, Bool)
```

- O tipo dos componentes é irrestrito

```
('a', (False, 'b')) :: (Char, (Bool, Char))
```

```
(True, ['a', 'b']) :: (Bool, [Char])
```

Tipos função

Uma **função** é um **mapeamento** de argumentos de um tipo para resultado de outro tipo

```
not :: Bool → Bool
```

```
even :: Int → Bool
```

Em geral

$t_1 \rightarrow t_2$ é o tipo de funções que mapeiam valores do tipo t_1 para valores para o tipo t_2

Nota

- A seta $\rightarrow$ é digitada no teclado como $\rightarrow$
- Os tipos de argumento e do resultado são **irrestritos**, por exemplo, funções com vários argumentos ou resultados são possíveis usando tuplas ou listas

```
add :: (Int,Int) → Int  
add (x,y) = x+y
```

```
zeroto :: Int → [Int]  
zeroto n = [0..n]
```

Funções *curried*

Funções com vários argumentos também são possíveis retornando **funções como resultados**

```
add' :: Int → (Int → Int)
add' x y = x+y
```

add' pega um inteiro **x** e retorna uma função **add' x**. Por sua vez, essa função pega um inteiro **y** e retorna o resultado **x+y**

Nota

- `add` e `add'` produzem o mesmo resultado final, mas `add` recebe seus dois argumentos ao mesmo tempo, enquanto `add'` os recebe um de cada vez

```
add :: (Int,Int) → Int
```

```
add' :: Int → (Int → Int)
```

- Funções que recebem um argumento um por vez são chamadas de *funções `curried`*, em homenagem ao trabalho de Haskell B. Curry em tais funções

- Funções com mais de dois argumentos podem ser *curried* retornando funções aninhadas

```
mult :: Int → (Int → (Int → Int))  
mult x y z = x*y*z
```

mult pega um inteiro **x** e retorna uma função **mult x**, que por sua vez toma um inteiro **y** e retorna uma função **mult x y**, que finalmente pega um inteiro **z** e retorna o resultado **x*y*z**.

Por que *currying* é útil?

As funções de *curried* são mais flexíveis do que as funções em tuplas, porque funções úteis muitas vezes podem ser feitas **aplicando parcialmente** uma função *curried*

Por exemplo

```
add'_1 :: Int → Int
```

```
take_5 :: [Int] → [Int]
```

```
drop_5 :: [Int] → [Int]
```


Convenções de *Currying*

Para evitar parênteses excessivos ao usar funções *curried*, duas convenções simples são adotadas

- A seta $\rightarrow$ associa-se à **direita**

$\text{Int} \rightarrow \text{Int} \rightarrow \text{Int} \rightarrow \text{Int}$

Significa $\text{Int} \rightarrow (\text{Int} \rightarrow (\text{Int} \rightarrow \text{Int}))$

- Como consequência, é natural que a aplicação da função se associe à **esquerda**

`mult x y z`

Significa `((mult x) y) z`

A menos que tuplas sejam explicitamente necessárias, todas as funções em Haskell são normalmente definidas na forma *curried*

Funções polimórficas

Uma função é chamada **polimórfica** ("de muitas formas") se seu tipo contém uma ou mais variáveis de tipo

`length :: [a] → Int`

Para qualquer tipo **a**, **length** usa uma lista de valores do tipo **a** e retorna um inteiro

Nota

- Variáveis de tipo podem ser **instanciadas** para tipos distintos em diferentes circunstâncias

```
> length [False,True]  
2
```

a = Bool

```
> length [1,2,3,4]  
4
```

a = Int

- Variáveis de tipo devem iniciar com uma letra minúscula e em geral são nomeadas a, b, c, etc.

- Muitas das funções definidas no padrão prelude são polimórficas, por exemplo

```
fst :: (a,b) → a
```

```
head :: [a] → a
```

```
take :: Int → [a] → [a]
```

```
zip :: [a] → [b] → [(a,b)]
```

```
id :: a → a
```

Funções sobrecarregadas

Uma função polimórfica é dita **sobrecarregada** se seu tipo contiver uma ou mais **restrições de classe**

$$(+)\ ::\ \text{Num } a \Rightarrow a \rightarrow a \rightarrow a$$

Para qualquer tipo numérico **a**, **(+)** usa dois valores do tipo **a** e retorna um valor do tipo **a**

Nota

- Variáveis com restrições de classe podem ser instanciadas para qualquer tipo que as satisfaçam

```
> 1 + 2  
3
```

a = Int

```
> 1.0 + 2.0  
3.0
```

a = Float

```
> 'a' + 'b'  
ERROR
```

Char não é um
tipo numérico

- Haskell tem uma série de classes de tipos, incluindo

Num - Tipos numéricos

Eq - Tipos igualdade

Ord - Tipos ordenados

z Por exemplo

```
(+) :: Num a => a -> a -> a
```

```
(==) :: Eq a => a -> a -> Bool
```

```
(<) :: Ord a => a -> a -> Bool
```


Dicas e sugestões

- Ao definir uma nova função em Haskell, é útil começar definindo seu tipo
- Em um script, é uma boa prática indicar o tipo de cada nova função definida
- Ao indicar os tipos de funções polimórficas que usam números, igualdade ou ordenações, tenha o cuidado de incluir as restrições de classe necessárias

Exercícios

(1) Quais são os tipos dos seguintes valores?

```
['a', 'b', 'c']
```

```
('a', 'b', 'c')
```

```
[(False, '0'), (True, '1')]
```

```
([False, True], ['0', '1'])
```

```
[tail, init, reverse]
```

Exercícios

(2) Escreva definições que têm os seguintes tipos, não importando o que elas realmente fazem, desde que estejam corretas

```
bools :: [Bool]
```

```
nums  :: [[Int]]
```

```
add   :: Int -> Int -> Int -> Int
```

```
copy  :: a -> (a,a)
```

```
apply :: (a -> b) -> a -> b
```

(3) Quais são os tipos das seguintes funções?

```
second xs = head (tail xs)
```

```
swap (x,y) = (y,x)
```

```
pair x y = (x,y)
```

```
double x = x*2
```

```
palindrome xs = reverse xs == xs
```

```
twice f x = f (f x)
```

(4) Verifique suas respostas usando o GHCi

Exercícios

- (5) Por que não é viável, em geral, que os tipos de função sejam instâncias da classe Eq? Quando é viável? Dica: duas funções do mesmo tipo são iguais se sempre retornarem resultados iguais para argumentos iguais.